Lab Report Cover Page (Please print this and attach to the cover of your lab report)

Complex Engineering Problem	
CEP NAME	**Digital System Design For Image Convolution**
SUBJECT:	**Digital Systems Design**
NAME:	**Muhammad Umair Khan**
REGISTRATION:	**CIIT/FA23-BCE-084/ISB**
SUBMITTED TO:	**Mr. Muhammad Bilal Qasim**
DATE:	**20th May 2026**

Lab Assessment (To be filled by the lab-instructor)			
	In Lab Evaluation	/ 7	/10
Lab Report	Data Analysis	/ 1	
	Data Presentation	/ 1	
	Writing Style	/ 1	

Comments:

Instructor Name _____ Instructor Signature _____

Report (To be submitted by the student)

Students are required to submit a report on the lab comprising the following three sections:

1. A critical analysis of the lab activity
2. Summary of skills learned
3. Conclusion

Note: Keep your report concise and to-the-point.

Digital System Design for image convolution

Introduction:

This document describes the design and implementation of a real-time Sobel edge detector system implemented on a Spartan-3E FPGA using the Nexys2 board. This system takes an image of size 128x128 pixels from internal Block RAM, applies edge detection, and displays the result on the VGA monitor at a clock rate of 25 MHz . Edge detection is one of the most basic tasks involved in image processing, which involves finding edges or boundaries in the images by detecting sudden variations in the intensity values of pixels. Among many edge detectors available, the Sobel edge detector was chosen because of its best possible combination of complexity and accuracy of the detection process, involving addition, subtraction, and bit shifting only without the use of multipliers. The system is designed to work as a streaming pipeline, where it receives input pixels, applies a Sobel kernel to a 3x3 sliding window of the image through line buffers, calculates gradient magnitude, writes into output memory, and displays it on VGA.

Literature Review:

The report discusses the design and implementation of a real-time Sobel edge detector system that has been designed using Spartan-3E FPGA and Nexys2 board. This system receives an image of dimensions 128x128 pixels from the internal block RAM, processes it with edge detection algorithms and displays the processed image on the VGA display at a clock frequency of 25 MHz's Edge detection is one of the simplest operations performed while processing images, wherein edges or boundaries of the image can be detected by identifying any sudden change in pixel intensity values. Various edge detection algorithms exist, but Sobel edge detection algorithm has been used in our design owing to its optimal combination of complexity and efficiency of edge detection process that only uses addition, subtraction and bit shifting operation and not multiplication operation. The design can be visualized as a streaming pipeline that performs edge detection in stages by receiving pixels, applying Sobel kernel to a 3x3 sliding window of the image, calculating gradient magnitude, storing the data in output memory and displaying it on VGA.

The reason for limiting the processing of the image to 128×128 pixels grayscale image stems from the resource limitation of the Spartan-3E FPGA. For 128×128 pixels 8-bit grayscale image, the memory required is precisely 16 KB, allowing room not only for the input image and the output image (32 KB total) but also for the line buffer and other logic. An image size of 256×256 pixels will demand 64 KB of memory per image (128 KB total). This will exceed the on-chip memory availability, necessitating complicated off-chip memory access resulting in latency. Line buffer module holds two rows of 128 pixels. This will require 2,048 bits for each buffer, thus, easily handled using distributed RAM. The use of grayscale image rather than the color image saves the requirement of dealing with three different RGB channels, saving memory requirement by 66% and arithmetic logic by 66%.A 128x128 resolution also allows for easy address calculation using the equation (y << 7) + x, where a left shift of 7 bits is equivalent to multiplying by 128. The

resolution gives clear visual output when displayed on VGA, operates in about 0.66 milliseconds per frame at 25 MHz, and ensures the system remains testable and debuggable, thus suitable for educational and embedded vision purposes.

Verilog Code:

Clock Divider Module:

```
module clk_div_25 (
    input  wire clk_50,
    input  wire reset,
    output reg  clk_25
);
    always @(posedge clk_50 or posedge reset) begin
        if (reset)
            clk_25 <= 0;
        else
            clk_25 <= ~clk_25;
    end
endmodule
```

VGA timing Module:

```
module vga_timing (
    input  wire clk,    input  wire reset,    output wire [9:0] x,
    output wire [8:0] y,    output wire hsync,    output wire vsync,
    output wire display_active
);
    parameter H_VISIBLE = 640;
    parameter H_FRONT   = 16;
    parameter H_SYNC    = 96;
    parameter H_BACK    = 48;
    parameter H_TOTAL   = 800;
    parameter V_VISIBLE = 480;
    parameter V_FRONT   = 10;
    parameter V_SYNC    = 2;
    parameter V_BACK    = 33;
    parameter V_TOTAL   = 525;
    reg [9:0] h_count;    reg [9:0] v_count;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            h_count <= 0;
            v_count <= 0;
        end
        else begin
            if (h_count == H_TOTAL - 1) begin
                h_count <= 0;
                if (v_count == V_TOTAL - 1)
                    v_count <= 0;
                else
                    v_count <= v_count + 1;
            end
            else
                h_count <= h_count + 1;
        end
```

```
    end
    assign x = (h_count < H_VISIBLE) ? h_count : 0;
    assign y = (v_count < V_VISIBLE) ? v_count : 0;
    assign hsync = ~((h_count >= (H_VISIBLE + H_FRONT)) &&
                     (h_count <  (H_VISIBLE + H_FRONT + H_SYNC)));
    assign vsync = ~((v_count >= (V_VISIBLE + V_FRONT)) &&
                     (v_count <  (V_VISIBLE + V_FRONT + V_SYNC)));
    assign display_active = (h_count < H_VISIBLE) && (v_count < V_VISIBLE);
endmodule
```

Line Buffer Module:

```
module line_buffer (
    input  wire clk,    input  wire reset,    input  wire valid,
    input  wire [7:0] pixel_in,    output reg [7:0] p00,p01,p02,
    output reg [7:0] p10,p11,p12,    output reg [7:0] p20,p21,p22
);
    reg [7:0] line1 [0:127];    reg [7:0] line2 [0:127];    reg [9:0] col;
    integer i;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            col <= 0;
            for (i=0; i<128; i=i+1) begin
                line1[i] <= 0;
                line2[i] <= 0;
            end
            {p00,p01,p02,p10,p11,p12,p20,p21,p22} <= 0;
        end
        else if (valid) begin
            line2[col] <= line1[col];
            line1[col] <= pixel_in;
            p22 <= line2[col];
            p21 <= (col > 0) ? line2[col-1] : 0;
            p20 <= (col > 1) ? line2[col-2] : 0;
            p12 <= line1[col];
            p11 <= (col > 0) ? line1[col-1] : 0;
            p10 <= (col > 1) ? line1[col-2] : 0;
            p02 <= pixel_in;014
            p01 <= (col > 0) ? line1[col-1] : 0;
            p00 <= (col > 1) ? line1[col-2] : 0;
            if (col == 127) begin
                col <= 0;
            end else begin
                col <= col + 1;
            end
        end
    end
endmodule
```

Sobel Core:

```
module SobelCore(
    input  wire clk,
     input  wire reset,
     input  wire valid,
```

```
    input  wire [7:0] w00,w01,w02,
    input  wire [7:0] w10,w11,w12,
    input  wire [7:0] w20,w21,w22,
    output reg signed [11:0] gx,
    output reg signed [11:0] gy
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            gx <= 0;
            gy <= 0;
        end
        else if (valid) begin
            gx <= (w02 + (w12<<1) + w22) - (w00 + (w10<<1) + w20);
            gy <= (w20 + (w21<<1) + w22)- (w00 + (w01<<1) + w02);
        end
    end
endmodule
```

Final Calculation Module:

```
module Sum(
    input  wire clk,
    input  wire reset,
    input  wire valid,
    input  wire signed [11:0] gx,
    input  wire signed [11:0] gy,
    output reg [7:0] edge_out,
    output reg out_valid
);
    reg [11:0] ax, ay;
    reg [12:0] sum;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            edge_out <= 0;
            out_valid <= 0;
        end
        else if(valid) begin
            ax = (gx < 0) ? -gx : gx;
            ay = (gy < 0) ? -gy : gy;
            sum = ax + ay;
            edge_out <= (sum > 255) ? 8'd255 : sum[7:0];
            out_valid <= 1;
        end
                        else
                        out_valid<=0;
    end
endmodule
```

Top Module:

```
module Top_Sobel(
    input  wire clk_50,
    input  wire reset,
    output wire hsync,
    output wire vsync,
    output wire [2:0] vga_r,
    output wire [2:0] vga_g,
    output wire [1:0] vga_b
);
    wire [7:0] pixel;  wire clk;
    clk_div_25 U0 (.clk_50(clk_50), .reset(reset), .clk_25(clk));
    wire [9:0] x;    wire [8:0] y;    wire display_active;
    vga_timing U2 (
        .clk(clk), .reset(reset), .x(x), .y(y), .hsync(hsync),
        .vsync(vsync), .display_active(display_active)
    );
    wire process_active = display_active && (x < 128) && (y < 128);
    wire [13:0] addr_a;
    assign addr_a = (y << 7) + x;
    wire [7:0] pixel_a;
    B1 B1 (
            .clka(clk), // input clka
            .wea(1'd0), // input [0 : 0] wea
            .addra(addr_a), // input [13 : 0] addra
            .dina(8'd0), // input [7 : 0] dina
            .douta(pixel_a) // output [7 : 0] douta
     );
    wire [7:0] p00,p01,p02,p10,p11,p12,p20,p21,p22;
    line_buffer LB (
        .clk(clk),
        .reset(reset),
        .pixel_in(pixel_a),
        .valid(process_active),
        .p00(p00),.p01(p01),.p02(p02),
        .p10(p10),.p11(p11),.p12(p12),
        .p20(p20),.p21(p21),.p22(p22)
    );
    wire signed [11:0] gx, gy;
    SobelCore S1 (
        .clk(clk),
        .reset(reset),
        .valid(process_active),
        .w00(p00),.w01(p01),.w02(p02),
        .w10(p10),.w11(p11),.w12(p12),
        .w20(p20),.w21(p21),.w22(p22),
        .gx(gx),
        .gy(gy)
    );
    wire [7:0] pixel_out;
    wire valid2;
    Sum S2 (
        .clk(clk),
        .reset(reset),
        .valid(process_active),
```

```
        .gx(gx),
        .gy(gy),
        .edge_out(pixel_out),
        .out_valid(valid2)
    );
    reg [13:0] write_addr;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            write_addr <= 0;
        end else if (valid2) begin
            write_addr <= write_addr + 1;
        end
    end
    wire [7:0] pixel_b;
    BlockRAM B2 (
      .clka(clk),
      .wea(valid2),
      .addra(write_addr), // Using the internal sequential counter
      .dina(pixel_out),
      .douta(pixel_b)
    );
    assign pixel = pixel_out;
    assign vga_r = process_active ? pixel_out[7:5] : 3'd0;
    assign vga_g = process_active ? pixel_out[4:2] : 3'd0;
    assign vga_b = process_active ? pixel_out[1:0] : 2'd0;
endmodule
```

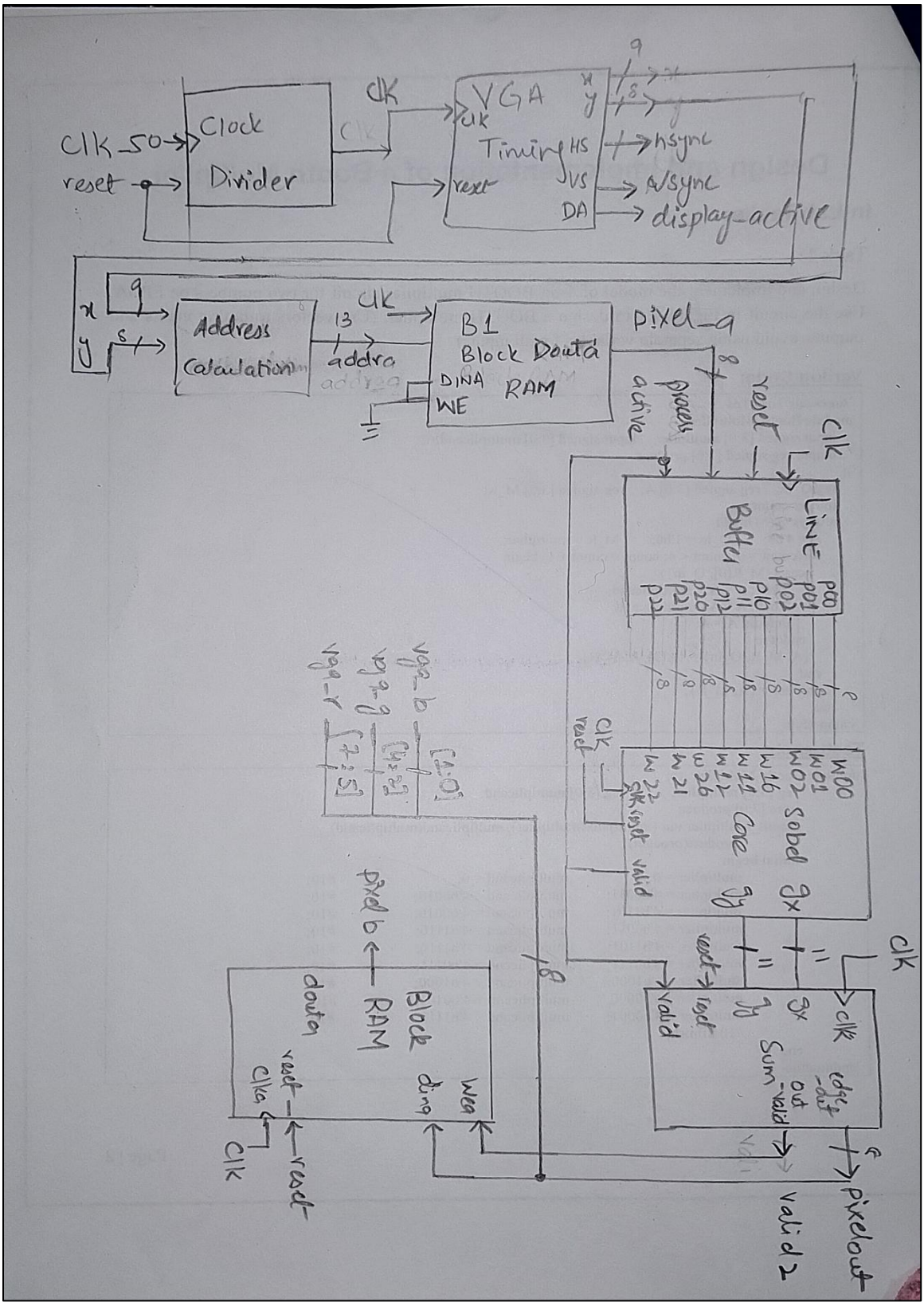
Block RAM:

Two block RAMS with the following instantiation templates were used:

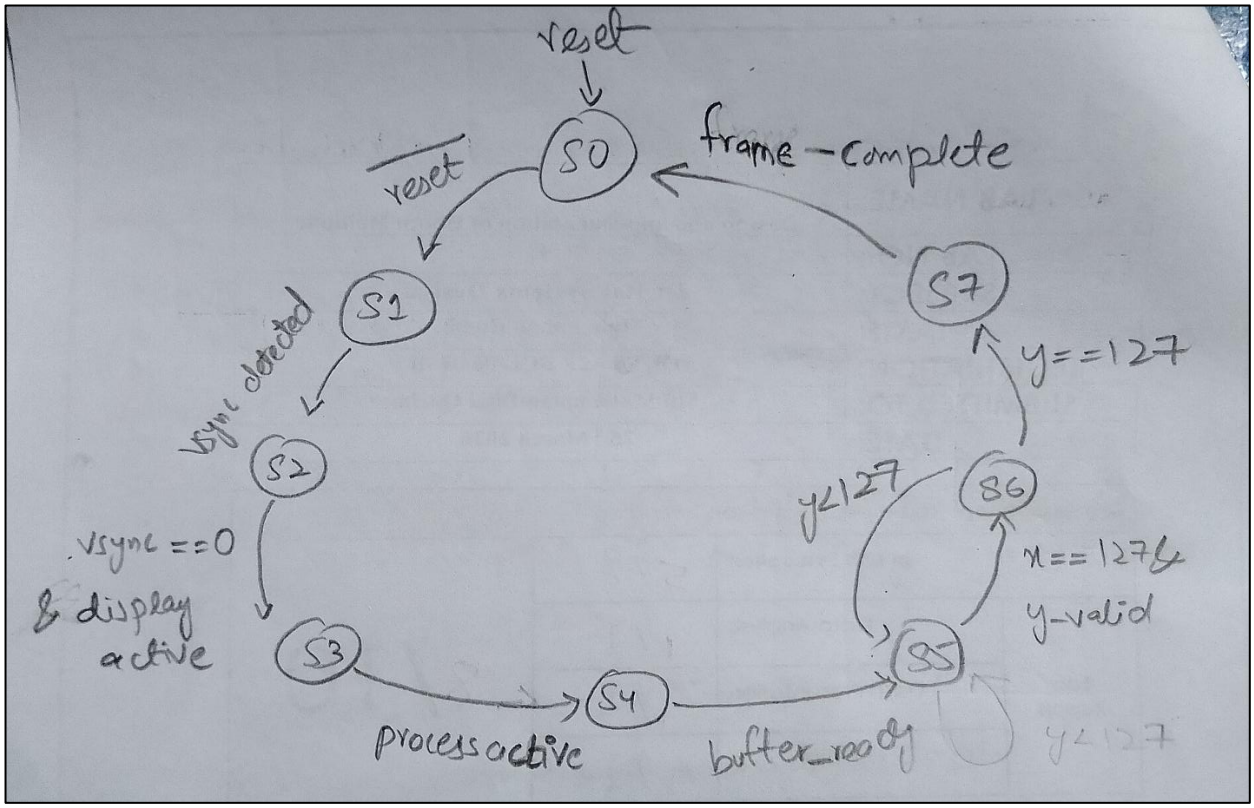
```
BlockRAM your_instance_name (
  .clka(clka), // input clka
  .wea(wea), // input [0 : 0] wea
  .addra(addra), // input [13 : 0] addra
  .dina(dina), // input [7 : 0] dina
  .douta(douta) // output [7 : 0] douta
);

B1 your_instance_name (
  .clka(clka), // input clka
  .wea(wea), // input [0 : 0] wea
  .addra(addra), // input [13 : 0] addra
  .dina(dina), // input [7 : 0] dina
  .douta(douta) // output [7 : 0] douta
);
```

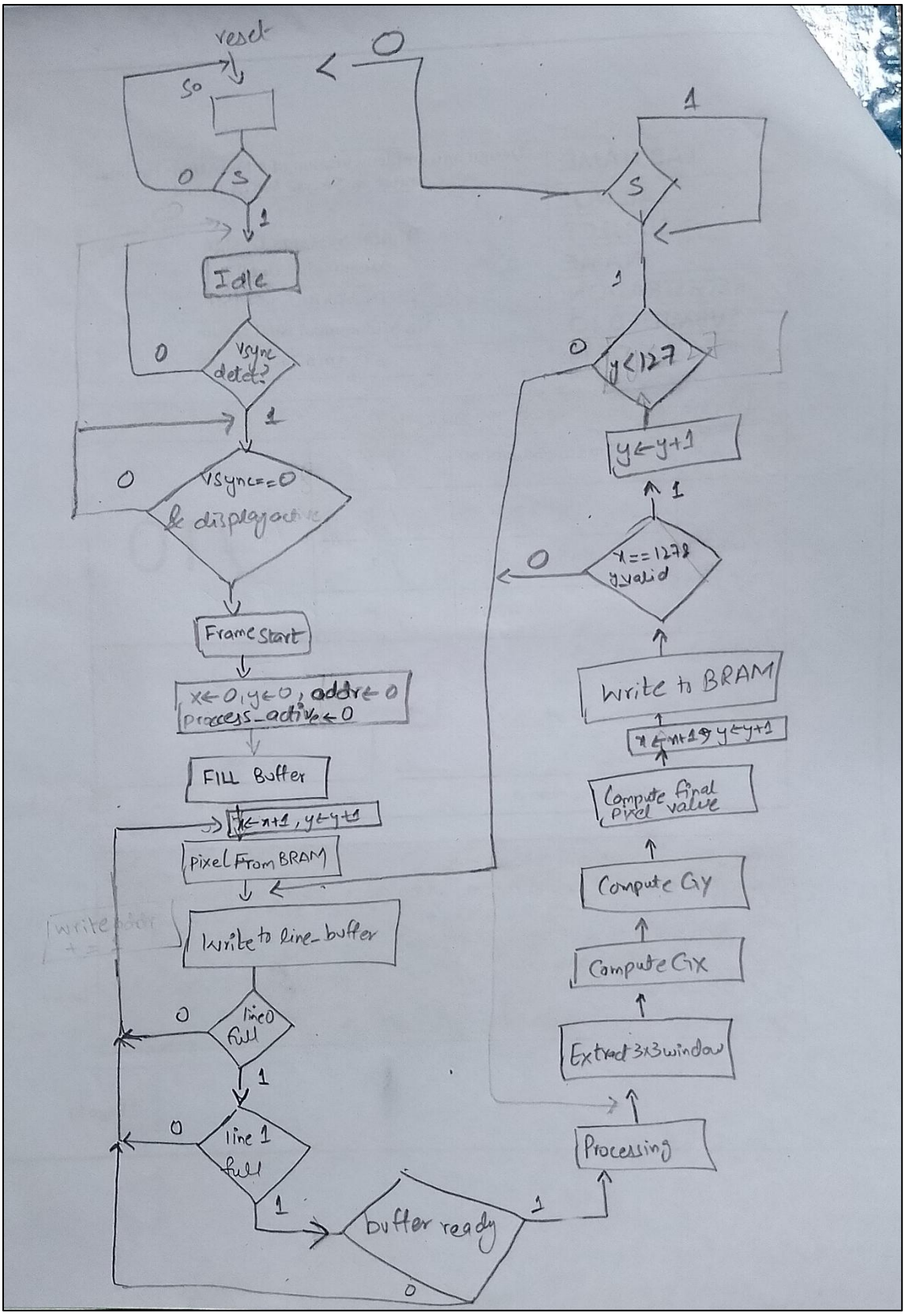
Block Diagram:



State Diagram:



ASM Chart:


reset
S0
S
Idle
vsync detect?
vsync==0 & displayactive
Frame Start
x←0, y←0, addr←0
process_active←0
FILL Buffer
x←x+1, y←y+1
pixel From BRAM
write to line_buffer
line0 full
line 1 full
buffer ready
Processing
Extract 3x3 window
Compute Gx
Compute Gy
Compute final pixel value
x←x+1, y←y+1
Write to BRAM
x==1278 y_valid
y<127
y←y+1

Filter Output:



Original Image



Filtered Image

Conclusion:

The Complex Engineering Problem involved designing and implementing a real-time Sobel edge detection system on a Spartan-3E FPGA using the Nexys2 development board. The system processed 128×128 grayscale images stored in Block RAM by using a 3×3 sliding window, line buffers, and VGA output operating at 25 MHz. The lab successfully showed that real-time image processing can be performed on an FPGA. The Sobel operator was a suitable choice because it is simple to implement in hardware and only requires addition, subtraction, and shifting instead of multipliers. The 128×128 image size was also practical because it matched the limited Block RAM resources of the Spartan-3E.

Despite the successful implementation, the design had some limitations. The system worked only with pre-loaded images and did not support live video input. Also, the top module displayed the pixel_out signal directly instead of properly reading the processed image from output Block RAM B2, so the second memory block was not fully used. Boundary pixels were handled using zero padding due to the 3×3 window, which may reduce accuracy near the image edges. In addition, there was no proper handshaking mechanism between pipeline stages, which could affect synchronization in larger designs.

Overall, this lab was a useful practical exercise in FPGA-based image processing. It improved understanding of pipeline design, memory interfacing, Sobel edge detection, line buffering, and VGA display generation. The project can be further improved by adding live video input, using the output Block RAM correctly, improving edge handling, and adding adjustable thresholding controls.